Module code: CS4001NP

Module Title: Programming

2026 Spring Module Leader: Sushil Parajuli

Anuj Thapa (25038629)

Assignment submission date: 30 June 2026

Declaration

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as no submission and a mark of zero will be awarded.

Table of Contents

Table of Figures

1.Introduction

NOC LPG Cylinder Management System is a Java based application which manages the records of domestic and commercial LPG cylinders. The main objective of this system is to help in managing the LPG cylinders. In this system the user can add cylinder records, calculate prices, identify types of cylinders, display records and save or load data from a file.

Java Object-Oriented Programming concepts were used to develop the system. The main concepts used in this project are Abstraction, Encapsulation, Inheritance and Polymorphism.

The application has a simple Graphical User Interface (GUI) made using Java Swing. The GUI consists of text fields, combo boxes, buttons and a text area for results. The interface was kept simple as the focus of the project is to demonstrate Java programming and Object-Oriented Programming concepts.

The system has two types of LPG cylinders:

- Domestic Cylinder
- Commercial Cylinder

Domestic cylinders have a subsidy amount and a citizenship number. Commercial cylinders must be labelled with quantity and business licence number. Before a new cylinder is added, the system also checks other validation rules.

The application stores the cylinder objects in an ArrayList<LPGCylinder>. This means that both the domestic and commercial cylinder objects can be stored in the same list because both inherit from the LPGCylinder abstract class.

2.Objective

This project is designed to develop an easy LPG cylinder management application using Java and Object-Oriented Programming concepts.

The specific objectives are:

1. Developing a Java application for maintaining LPG cylinder records.
2. To make an abstract class LPGCylinder which will have common details for cylinder.
3. Using inheritance, create classes DomesticCylinder and CommercialCylinder.
4. To demonstrate abstraction via an abstract base class.
5. To demonstrate encapsulation with private attributes and getter and setter methods.
6. To demonstrate inheritance by extending the class LPGCylinder.
7. To determine the final price of the local cylinder after a subsidy has been implemented.
8. To calculate the final selling price after the quantity discount is applied.
9. To avoid having the same Cylinder ID and Booking ID.
10. For validation of user inputs and handling wrong information.
11. To understand better Java classes, methods, inheritance, abstraction, encapsulation, polymorphism, GUI development, validation, and file management.

3. Class Diagram

Figure 1: Class Diagram

4.Method Description

4.2 LPGCylinder Methods

Method Name	getBookingId()
Parameter type and name	none
Return type	String
Brief description	Returns the booking ID.
Internal Working	Returns the private bookingId field directly.

Method Name	getCylinderId()
Parameter type and name	none
Return type	String
Brief description	Returns the cylinder ID.
Internal Working	Returns the private cylinderId field directly.

Method Name	getCylinderId()
Parameter type and name	none
Return type	String
Brief description	Returns the cylinder ID.
Internal Working	Returns the private cylinderId field directly.

Method Name	getCylinderType()
Parameter type and name	none
Return type	String
Brief description	Returns the cylinder type
Internal Working	Returns the private cylinderType field directly.

Method Name	getBasePrice()
Parameter type and name	none
Return type	double
Brief description	Returns the base price.
Internal Working	Returns the private basePrice field directly.

Method Name	getWeight()
Parameter type and name	none
Return type	double
Brief description	Returns the weight.
Internal Working	Returns the private weight field directly.

Method Name	getBookingMonth()
Parameter type and name	none
Return type	String
Brief description	Returns the booking month.
Internal Working	Returns the private bookingMonth field directly.

Method Name	setBookingId(String)
Parameter type and name	String bookingId
Return type	void
Brief description	Sets booking ID after checking it is not empty.
Internal Working	Checks if null or blank. Prints error if so, otherwise assigns.

Method Name	setCylinderId(String)
Parameter type and name	String cylinderId
Return type	void
Brief description	Sets cylinder ID after validating the NOC- format.
Internal Working	Verifies the string starts with NOC- and all trailing characters are digits. Rejects and prints error if either check fails.

Method Name	setCylinderType(String)
Parameter type and name	String cylinderType
Return type	void
Brief description	Sets cylinder type if not empty.
Internal Working	Checks for null or blank before assigning.

Method Name	setBasePrice(double)
Parameter type and name	double basePrice
Return type	void
Brief description	Sets price only if greater than zero.
Internal Working	Compares value to zero. Prints error if not positive.

Method Name	setWeight(double)
Parameter type and name	double weight
Return type	void
Brief description	Sets weight only if greater than zero
Internal Working	Compares value to zero. Prints error if not positive.

Method Name	setBookingMonth(String)
Parameter type and name	String bookingMonth
Return type	void
Brief description	Sets booking month if not empty..
Internal Working	Checks for null or blank before assigning.

Method Name	calculateFinalPrice()
Parameter type and name	None
Return type	double
Brief description	Abstract – child classes provide implementation.
Internal Working	No body in this class. Declared abstract to force child classes to override it.

4.3 DomesticCylinder Methods

Method Name	getSubsidyAmount()
Parameter type and name	none
Return type	double
Brief description	Returns the subsidy amount.
Internal Working	Returns the private subsidyAmount field directly.

Method Name	getCitizenshipNumber()
Parameter type and name	none
Return type	String
Brief description	Returns the citizenship number.
Internal Working	Returns the private citizenshipNumber field directly

Method Name	setSubsidyAmount(double)
Parameter type and name	double subsidyAmount
Return type	void
Brief description	Sets subsidy if not negative and not above base price.
Internal Working	Checks value >= 0 and value <= getBasePrice(). Prints error and returns without setting if invalid.

Method Name	setCitizenshipNumber(String)
Parameter type and name	String citizenshipNumber
Return type	void
Brief description	Sets citizenship number only if it passes validation.
Internal Working	Calls validateCitizenshipNumber(). Only assigns the field if the method returns true.

Method Name	validateCitizenshipNumber(String)
Parameter type and name	String cn
Return type	boolean
Brief description	Returns true if the citizenship number is exactly 12 numeric digits.
Internal Working	Returns false if the string is null, if length != 12, or if any character is not a digit (checked with Character.isDigit()). Returns true only if all checks passes.

Method Name	calculateFinalPrice()
Parameter type and name	none
Return type	double
Brief description	Returns base price minus the subsidy amount.
Internal Working	Subtracts subsidyAmount from getBasePrice(). If result is negative, returns 0 to prevent negative prices.

Method Name	display()
Parameter type and name	none
Return type	void
Brief description	Prints all domestic cylinder details to the console.
Internal Working	Calls all getter methods and prints formatted output including booking ID, cylinder ID, type, weight, base price, month, citizenship number, subsidy amount, and calculated final price.

4.3 CommercialCylinder Methods

Method Name	getBusinessLicense()
Parameter type and name	none
Return type	String
Brief description	Returns the business license number.
Internal Working	Returns the private businessLicense field directly.

Method Name	getQuantity()
Parameter type and name	none
Return type	int
Brief description	Returns the order quantity.
Internal Working	Returns the private quantity field directly.

Method Name	setBusinessLicense(String)
Parameter type and name	String businessLicense
Return type	void
Brief description	Sets business license if not empty.
Internal Working	Checks for null or blank. Prints error and returns without setting if invalid.

Method Name	setQuantity(int)
Parameter type and name	int quantity
Return type	void
Brief description	Sets quantity if it is a positive number.
Internal Working	Checks quantity > 0. Prints error message if the value is not positive.

Method Name	calculateFinalPrice()
Parameter type and name	none
Return type	double
Brief description	Returns total price after applying the applicable bulk discount.
Internal Working	Computes total = basePrice * quantity. If quantity >= 10, applies 5% discount. If >= 5, applies 3%. Otherwise no discount. Returns total minus discount amount.

Method Name	getDiscountRate()
Parameter type and name	none
Return type	double
Brief description	Returns the discount percentage (5.0, 3.0, or 0.0).
Internal Working	Uses the same if-else condition as calculateFinalPrice() to determine and return the applicable discount rate. Avoids repeating the logic in display().

Method Name	display()
Parameter type and name	none
Return type	void
Brief description	Prints all commercial cylinder details including discount info.
Internal Working	Calls getDiscountRate() to determine the rate, computes pre-discount total, and prints booking ID, cylinder ID, type, weight, base price, month, license, quantity, total before discount, discount rate, discount amount, and final price.

5.Pseudo Code

5.1 Adding Domestic Cylinder

START

Get selected month

Get Cylinder ID

Get Booking ID

Get Price

Get Weight

Get Subsidy Amount

Get Citizenship Number

IF any required field is empty

THEN Show error message

STOP

END IF

Check if Cylinder ID already exists

IF Cylinder ID already exists THEN

Show error message

STOP

END IF

Check if Booking ID already exists

IF Booking ID already exists THEN

Show error message

STOP

END IF

```
    Check citizenship number
    IF citizenship number is invalid THEN
        Show error message
        STOP
    END IF
    Count domestic cylinders for the same citizenship and same month
    IF count is 2 or more THEN
        Show quota exceeded error
        STOP
    END IF
    Convert price into number
    IF price is invalid or less than or equal to zero THEN
        Show error message
        STOP
    END IF
    Convert subsidy into number
    IF subsidy is negative THEN
        Show error message
    STOP
    END IF
    Create DomesticCylinder object
    Add DomesticCylinder to ArrayList
    Display success message
END
```

5.2 Adding Commerical Cylinder

START

Get selected month

Get Cylinder ID

Get Booking ID

Get Price

Get Weight

Get Quantity

Get Business License

IF any required field is empty THEN

Show error message

STOP

END IF

Check if Cylinder ID already exists

IF Cylinder ID already exists THEN

Show error message

STOP

END IF

Check if Booking ID already exists

IF Booking ID already exists THEN

Show error message

STOP

END IF

Convert price into number

IF price is invalid or less than or equal to zero THEN

Show error message

STOP

END IF

Convert quantity into integer

IF quantity is invalid or less than or equal to zero THEN

Show error message

STOP

END IF

Create CommercialCylinder object

Add CommercialCylinder to ArrayList

Display success message

END

5.3 Displaying All Cylinders

START

Check if ArrayList is empty

IF ArrayList is empty THEN

Display "No cylinders available"

STOP

END IF

Display heading

FOR each cylinder in ArrayList

Call display() method

Display cylinder information

END FOR

Display ending line

END

5.4 Calculating Domestic Final Price

START

Get Cylinder ID from search field

IF Cylinder ID is empty THEN

Display error

STOP

END IF

Search ArrayList using Cylinder ID

IF Cylinder ID is not found THEN

Display error

STOP

END IF

Check if object is DomesticCylinder

IF object is not DomesticCylinder THEN

Display "Use bulk discount instead"

STOP

END IF

Get base price

Get subsidy amount

Calculate: Final Price = Base Price - Subsidy Amount

Display final price

END

5.5Calculating Commercial Bulk Discount

START

Get Cylinder ID from search field

IF Cylinder ID is empty THEN

Display error

STOP

END IF

Search ArrayList using Cylinder ID

IF Cylinder ID is not found THEN

Display error

STOP

END IF

Check if object is CommercialCylinder

IF object is not CommercialCylinder THEN

Display "Use subsidy calculation instead"

STOP

END IF

Get base price

Get quantity

Calculate: Total Price = Base Price × Quantity

IF quantity >= 10 THEN

Discount = 5%

ELSE IF quantity >= 5 THEN

Discount = 3%

```
    ELSE
        Discount = 0%
    END IF
    Calculate final price
    Display final price
END
```

5.6 Domestic Monthly Quota

```
START
    Get Citizenship Number
    Get selected Month
    Set count = 0
    FOR each cylinder in ArrayList
        IF cylinder is DomesticCylinder THEN
            Check citizenship number
            Check month
            IF both are the same THEN
                Increase count by 1
            END IF
        END IF
    END FOR
    IF count >= 2 THEN
        Show "Domestic quota exceeded"
        Do not add new cylinder
    ELSE
        Allow domestic cylinder to be added
    END IF
END
```

5.6 NOCApp GUI Initialization

START

Create JFrame

Create Input Panel

Create Labels

Create Text Fields

Create Combo Boxes

Create Buttons

Create Output Text Area

Add all components to panels

Add panels to JFrame

Register Action Listeners for each button

Display JFrame

END

5. 7 Add Domestic Cylinder Button

START

User clicks "Add Domestic Cylinder"

Read all input fields

IF any field is empty THEN

Show error message

STOP

END IF

Check duplicate Cylinder ID

IF duplicate found THEN

Show error message

STOP

END IF

Check duplicate Booking ID

IF duplicate found THEN

Show error message

STOP

END IF

Check monthly quota

IF quota exceeded THEN

Show error message

STOP

END IF

Create DomesticCylinder object

Add object into ArrayList

Display success message

END

5.8 Add Commercial Cylinder Button

START

User clicks "Add Commercial Cylinder"

Read all input values

IF any field is empty THEN

Show error

STOP

END IF

Validate price

Validate quantity

Check duplicate IDs

Create CommercialCylinder object

Add object into ArrayList

Display success message

END

5.9 Display All Button

START

User clicks "Display All"

IF ArrayList is empty THEN

Display "No cylinders available"

ELSE

FOR each cylinder in ArrayList

Display cylinder information

END FOR

END IF

END

5.10 Calculate Domestic Price Button

START

Enter Cylinder ID

Click "Calculate Bulk Discount"

Search cylinder

IF cylinder found THEN

IF Commercial THEN

Calculate total price

Apply discount

Display result

ELSE

Show message

"Not a Commercial Cylinder"

END IF

ELSE

Show error

END IF

END

5.11 Calculate Commercial Discount Button

START

Enter Cylinder ID

Click "Calculate Bulk Discount"

Search cylinder

IF cylinder found THEN

IF Commercial THEN

Calculate total price

Apply discount

Display result

ELSE

Show message

"Not a Commercial Cylinder"

END IF

ELSE

Show error

END IF

END

5.12 Identify Cylinder Button

START

Enter Cylinder ID

Click "Identify Cylinder"

Search ArrayList

IF cylinder found THEN

IF Domestic

Display "Domestic Cylinder"

ELSE

Display "Commercial Cylinder"

END IF

ELSE

Display "Cylinder Not Found"

END IF

END

5.13 Clear Button

START

Click "Clear"

Clear all text fields

Clear search field

Display "Fields Cleared"

END

5.14 Export Button

START

Click "Export"

IF ArrayList is empty THEN

Show error

STOP

END IF

Open Save File Dialog

Create text file

Write all cylinder records

Close file

Display success message

END

5.15 Load Button

START

Click "Load"

Open File Dialog

Read text file

FOR each line

Create object

Add object into ArrayList

END FOR

Display loaded records

END

6. Testing Evidence

6.1 **Compile & Run via GUI.**

Test No.	01
Description	Program executes properly with proper GUI interface.
Steps/Input	Run the main file NOC app.
Expected result	Program starts successfully and GUI window opens.
Actual result	Program starts successfully and GUI window opens.
Result (Pass/Fail)	Pass

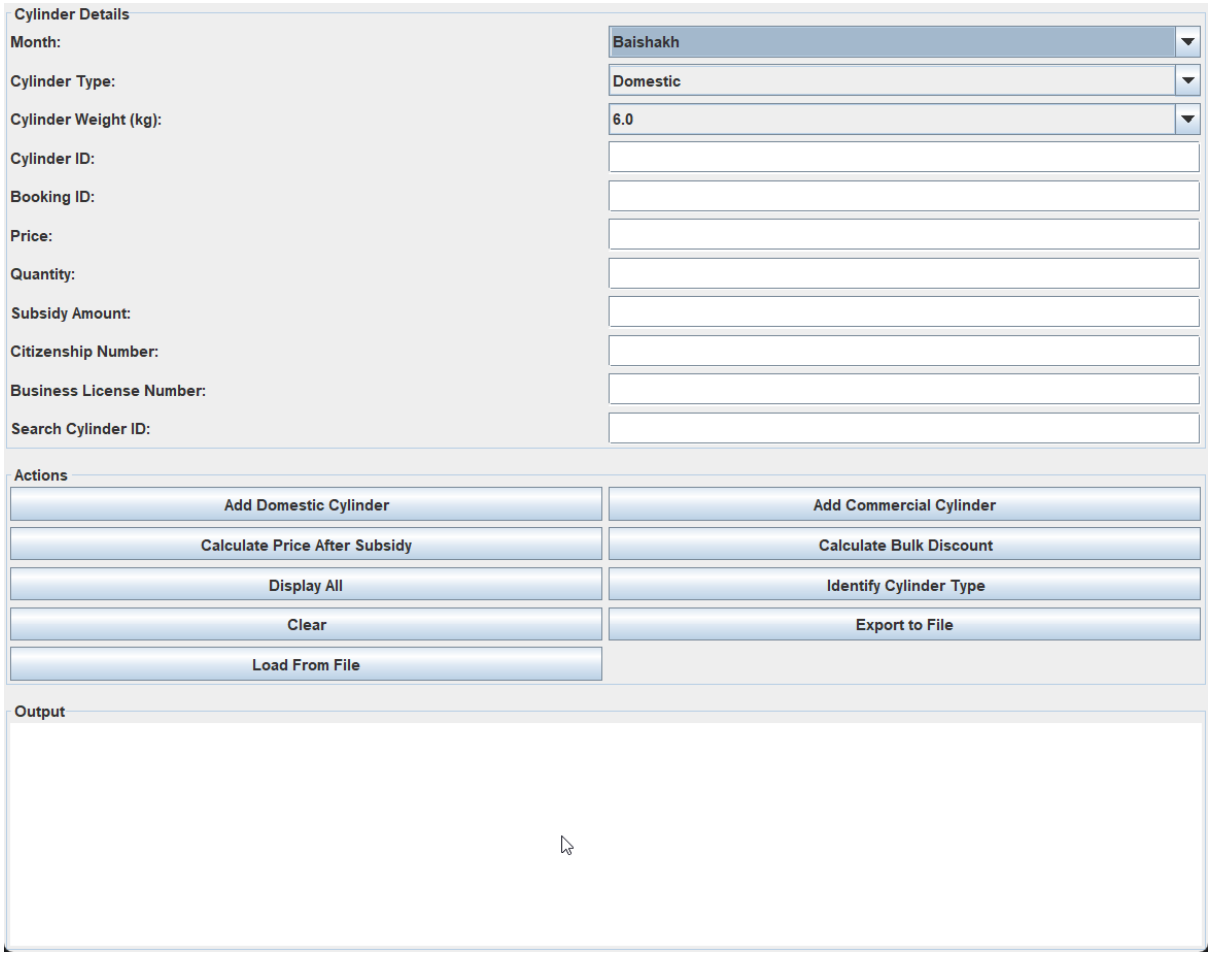

Figure 2:GUI working successfully

6.2 Adding Domestic Cylinder

Test No.	02
Description	Add a valid domestic cylinder record.
Steps/Input	Select a month, enter a valid Cylinder ID, Booking ID, Price, Weight, Subsidy Amount, Citizenship Number, and click Add Domestic Cylinder.
Expected result	Domestic cylinder is added successfully and displayed in the output area.
Actual result	Domestic cylinder is added successfully and displayed in the output area.
Result (Pass/Fail)	Pass

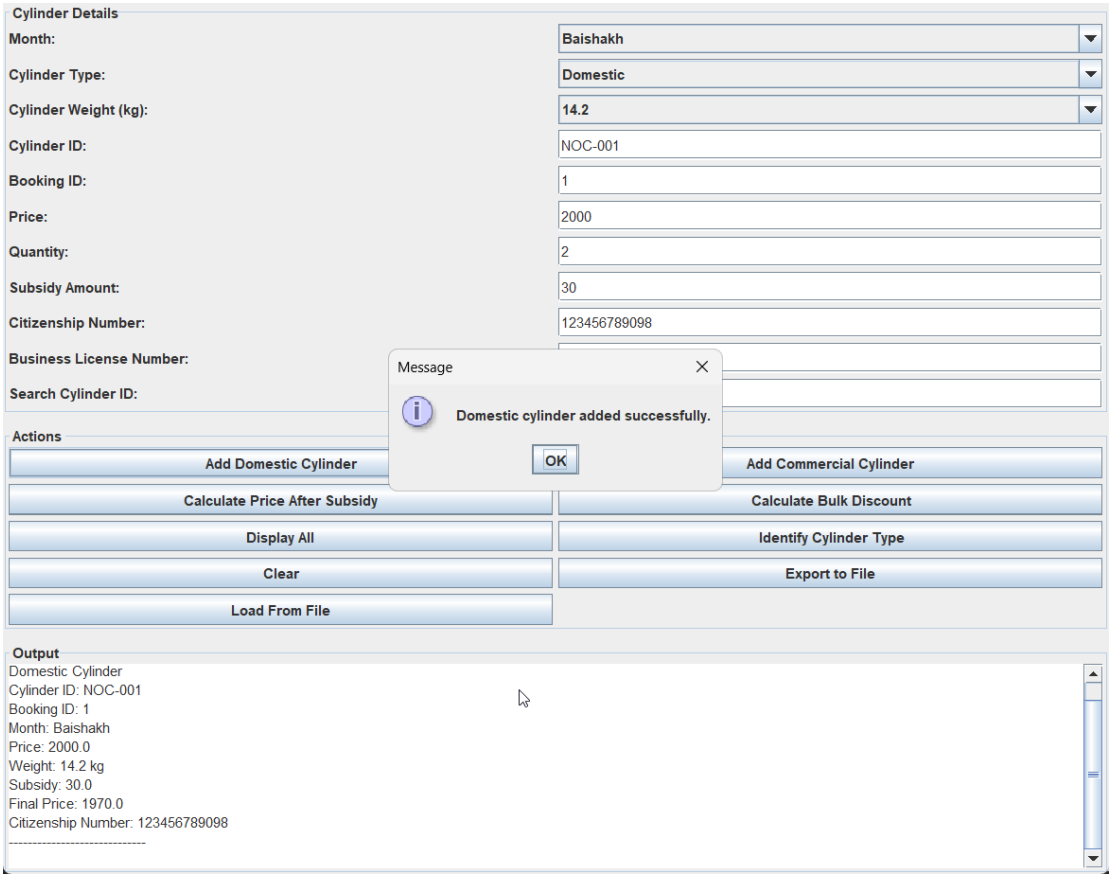

Figure 3:Adding Domestic Cylinder

6.3 Adding Commercial Cylinder

Test No.	03
Description	Add a valid commercial cylinder record.
Steps/Input	Select a month, enter a valid Cylinder ID, Booking ID, Price, Weight, Quantity, Business License, and click Add Commercial Cylinder.
Expected result	Commercial cylinder is added successfully and displayed in the output area.
Actual result	Commercial cylinder is added successfully and displayed in the output area.
Result (Pass/Fail)	Pass

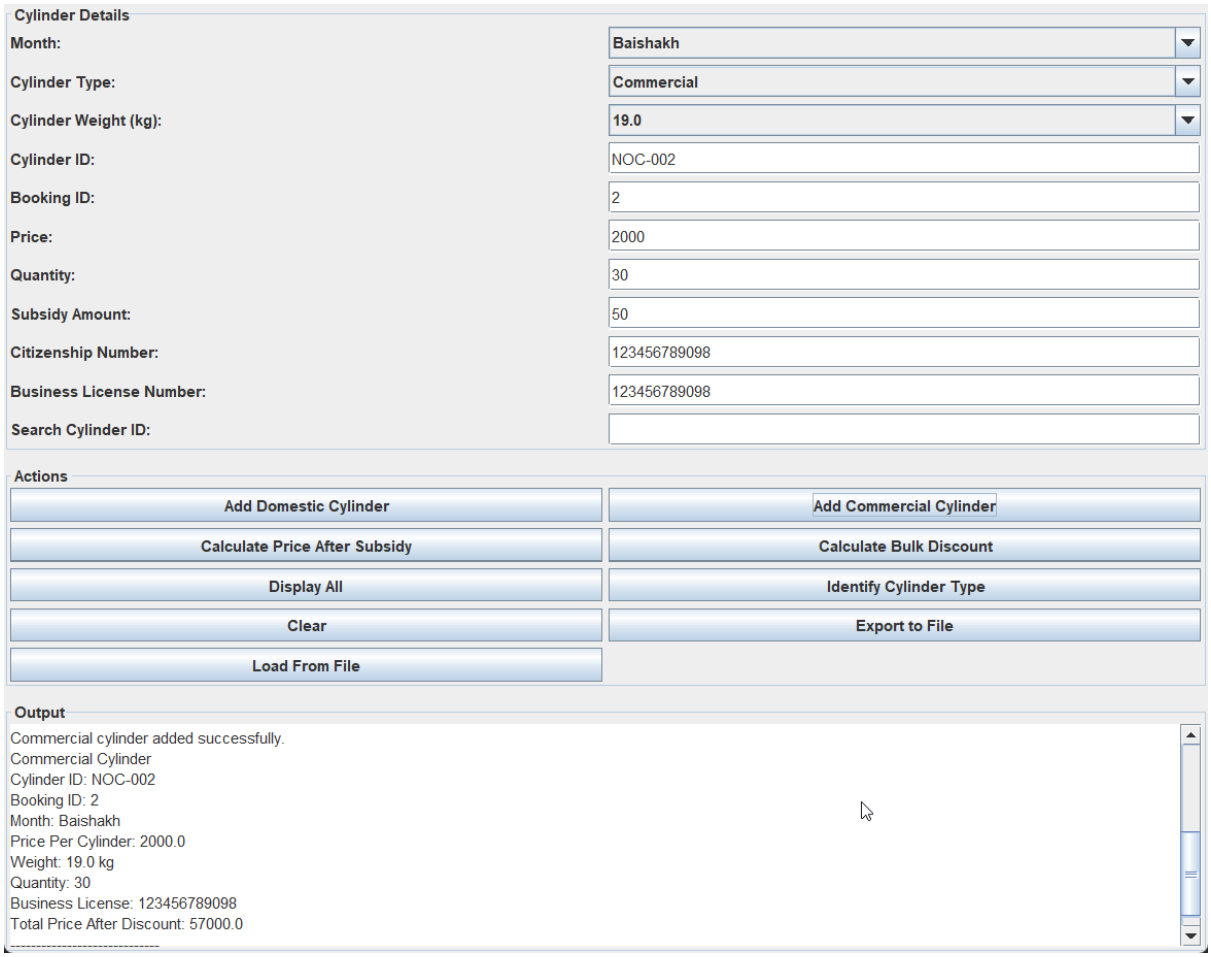

Figure 4:Adding Commercial Cylinder

6.4 Display all Records

Test No.	04
Description	Display all stored cylinder records.
Steps/Input	Click the Display All button after adding cylinder records.
Expected result	All stored domestic and commercial cylinder records are displayed in the output area.
Actual result	All stored domestic and commercial cylinder records are displayed in the output area.
Result (Pass/Fail)	Pass

Figure 5: Displayed all record in Output

6.5 Calculation of domestic price after Subsidy

Test No.	05
Description	Calculate domestic cylinder price after subsidy.
Steps/Input	Enter a valid domestic Cylinder ID in the search field and click Calculate Price After Subsidy.
Expected result	The final price after deducting the subsidy amount is calculated and displayed.
Actual result	The final price after deducting the subsidy amount is calculated and displayed.
Result (Pass/Fail)	Pass

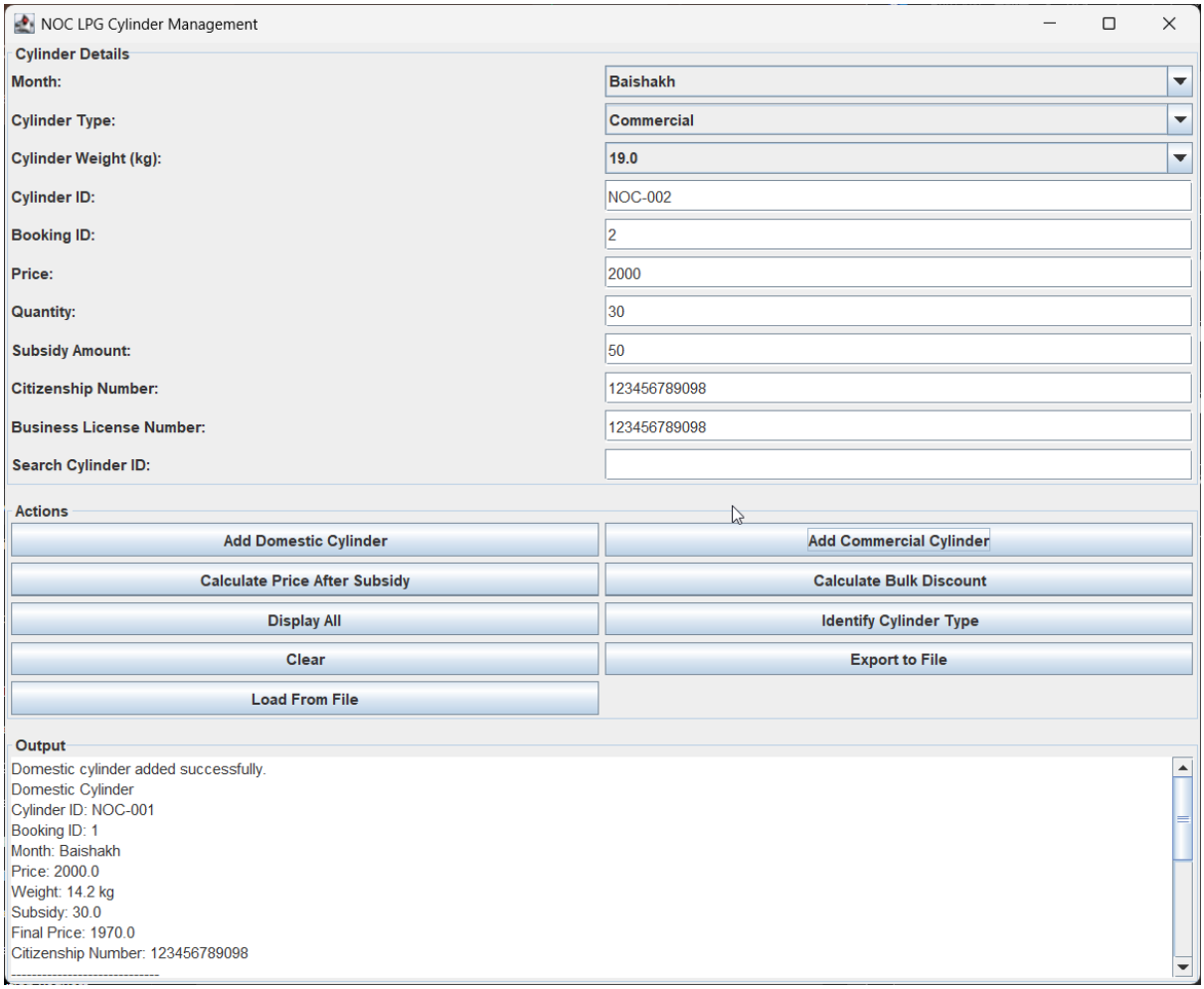

Figure 6: Calculate domestic cylinder price after subsidy.

6.6 Commercial Cylinder Bulk Discount

Test No.	06
Description	Calculate commercial cylinder bulk discount.
Steps/Input	Enter a valid commercial Cylinder ID in the search field and click Calculate Bulk Discount.
Expected result	The total price with the applicable bulk discount is calculated and displayed.
Actual result	The total price with the applicable bulk discount is calculated and displayed.
Result (Pass/Fail)	Pass

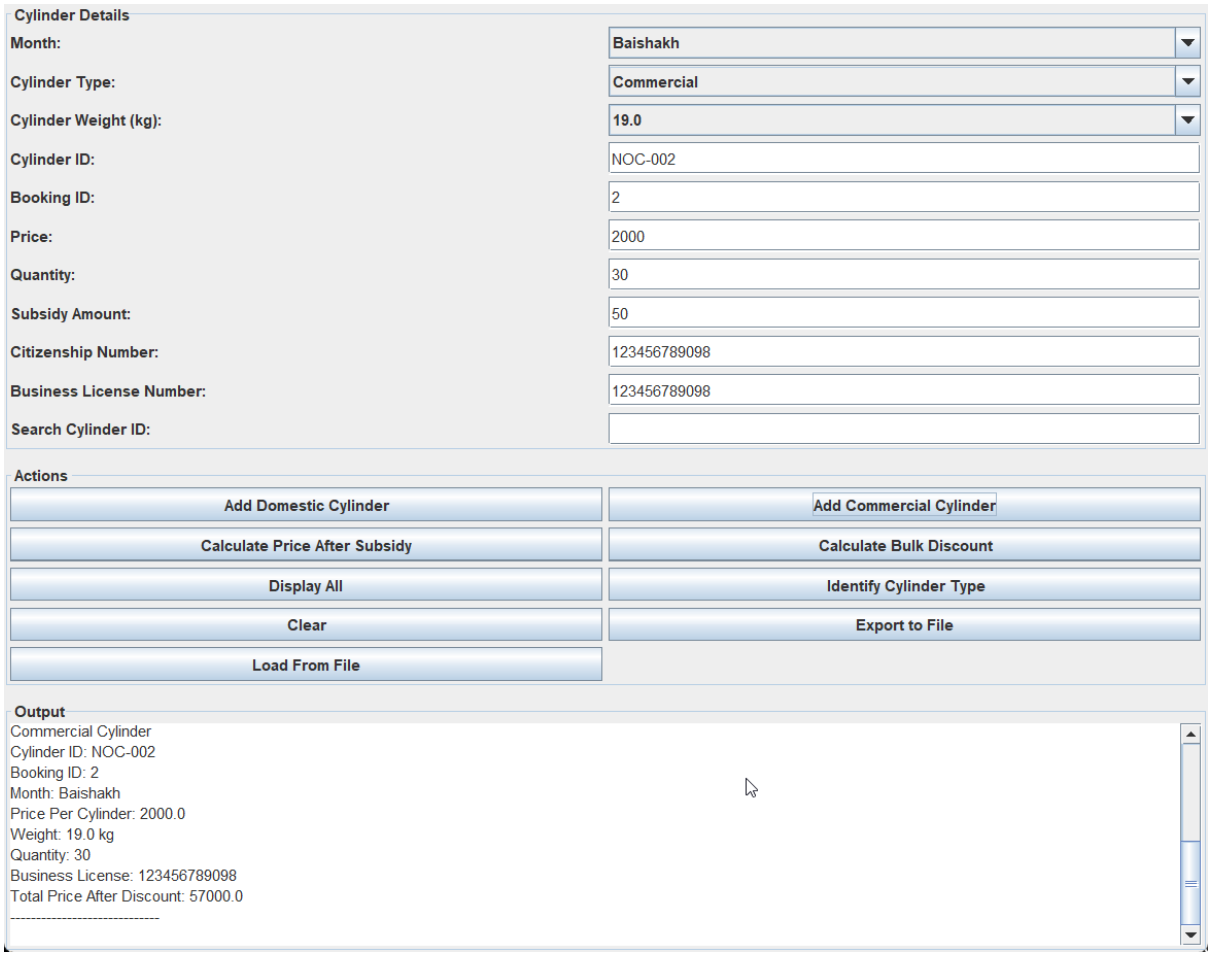

Figure 7: Bulk discount in Commercial Cylinder

6.7 Identify cylinder type

Test No.	07
Description	Identify cylinder type.
Steps/Input	Enter a valid Cylinder ID in the search field and click Identify Cylinder Type.
Expected result	The application correctly identifies whether the cylinder is Domestic or Commercial.
Actual result	The application correctly identifies whether the cylinder is Domestic or Commercial.
Result (Pass/Fail)	Pass

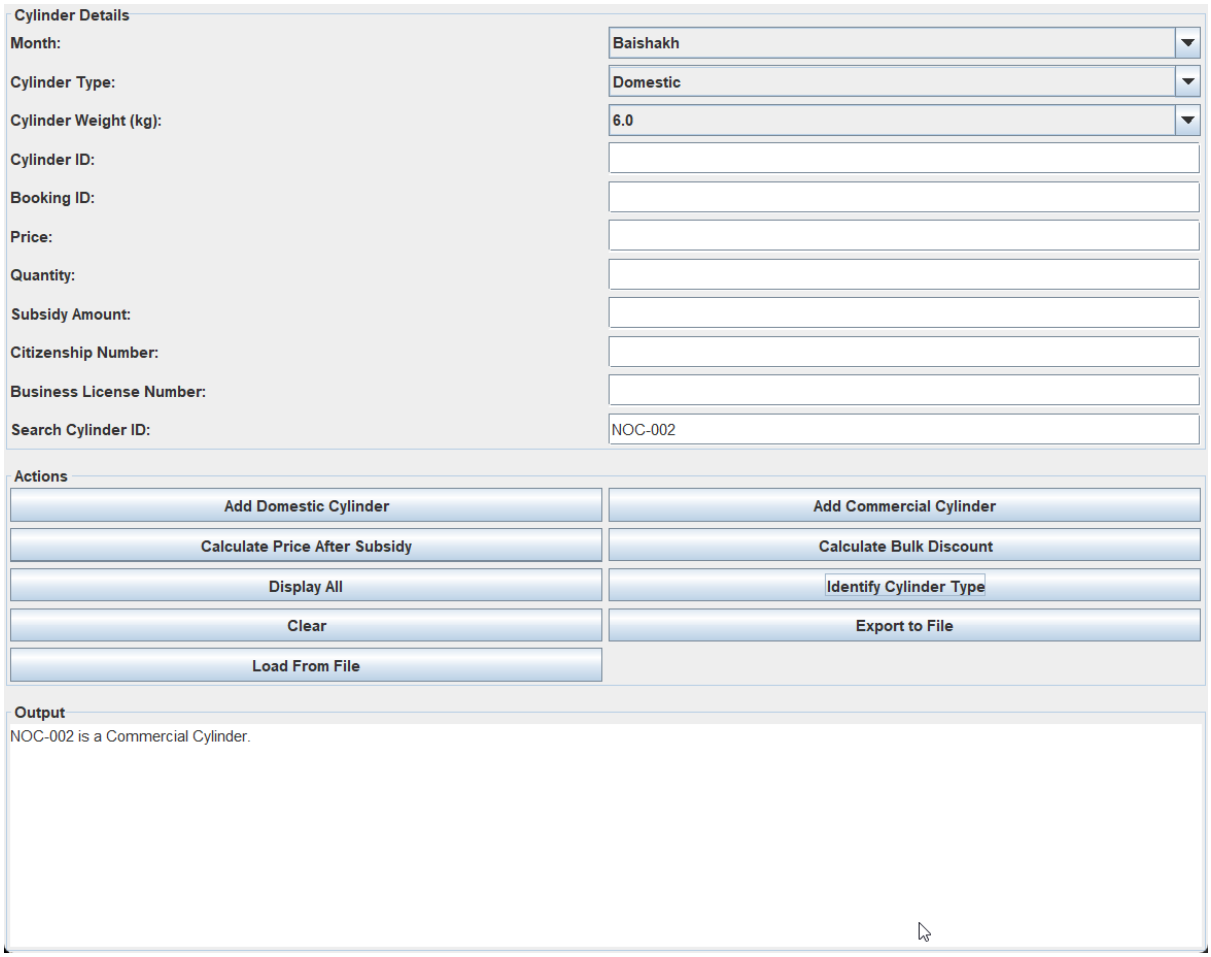

Figure 8:Identify cylinder type.

6.8 Validation

Test No.	08
Description	Validate required fields when adding a domestic cylinder.
Steps/Input	Open the application. Select Domestic as the cylinder type. Leave one or more required fields (Cylinder ID, Booking ID, Price, Subsidy Amount, or Citizenship Number) empty. Click the Add Domestic Cylinder button.
Expected result	The system displays an error message stating "Please fill all required domestic fields." The domestic cylinder record is not added to the system.
Actual result	The system displays the error message "Please fill all required domestic fields." and the record is not added.
Result (Pass/Fail)	Pass

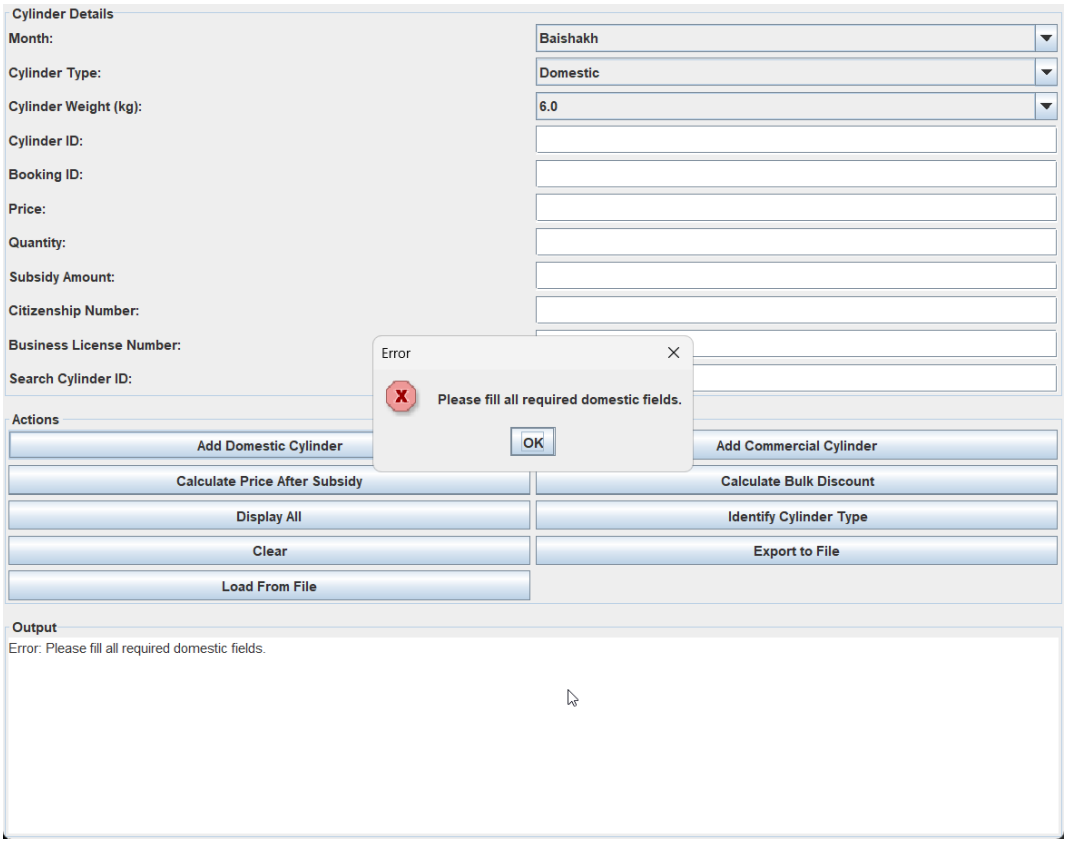

Figure 9: Validation

6. Error Detection and Correction

6.1 Syntax Error

Original code: calculateFinalPrice() was missing a return statement at the end when I was first creating the DomesticCylinder class. In the method I forgot to write a return statement, and it was supposed to return a double value.

Make sure to include a return statement in a method.Be sure to include the return statement in a method (DomesticCylinder.java:xx: error: missing return statement).

Buggy code:

```
public double calculateFinalPrice() {
    double finalPrice = getBasePrice() - subsidyAmount;
    if (finalPrice < 0) {
        finalPrice = 0;
    }
    // forgot to write: return finalPrice;
}
```

Fixed code:

```
public double calculateFinalPrice() {
    double finalPrice = getBasePrice()-subsidyAmount;
    if (finalPrice < 0) {
        finalPrice = 0;
    }
    finalPrice = "return finalPrice;";  // added this line
}
```

How I spotted it: javac compiler immediately pointed to the exact line number with the error message. I read it and realized that the return statement was not there.

6.2 Runtime Error

I was testing the program and when I added a null value as the bookingId to addDomesticCylinder(), the program crashed with a NullPointerException. This was done because I was calling .trim() on the bookingId before checking to see if it was null.

If this error occurs, restart the computer and repeat steps 1 through 4.If this error message appears, restart the computer and repeat steps 1-4.addDomesticCylinder(NOCApp.java:...)

Buggy code:

```
//This will crash if bookingId is null.
if (bookingId.trim().isEmpty()) {
    System.out.println("ERROR: Booking ID can not be empty!");
    return;
}
```

Fixed code:

```
If(bookingId == null || bookingId.trim().isEmpty()) {
    System.out.println("ERROR: Booking ID must not be blank!");
    return;
}
```

Where I found it: the line showing up in the terminal with a stack trace. I was taught to use null safety when calling methods on a string parameter.

6.3 Logical Error

The first version of the countDomesticByMonthAndCitizen() was missing the month condition in the comparison. This resulted in the quota being counted on all the

cylinders for all the months, not just the current month. Thus, if a customer booked in Baishakh and tried to book in Jestha, then he was getting the wrong block.

Buggy code:

```
//WRONG, it will only check the citizenship number and not the month.
if (dc.getCitizenshipNumber().equals(citizenshipNum)) {
    count++;
}
```

Fixed code:

```
if (dc.getCitizenshipNumber().equals(citizenshipNum)
        && dc.getBookingMonth().equals(month)) {
    count++;
}
```

How I discovered it: This was discovered when I was running Test T11a. The second booking in Baishakh was being rejected despite being a valid booking. I followed the count logic step by step and found that the month was never being compared.

7. Conclusion and Reflection

I really enjoyed completing Milestone 1. The development of the NOC LPG system enabled me to see how the four OOP concepts are operating in practice, and not just theory.

The most useful thing I learned was the use of the abstract class and inheritance to keep the code, organized! I could move the common rules into LPGCylinder and add the type-specific rules into the derived classes, thus avoiding the duplication of the validation logic in each class. This removed the repetition and thus made the code easier to read.

This work greatly helped me to understand polymorphism. I was pleased to store both DomesticCylinder and CommercialCylinder in an ArrayList and then call display()

through parent reference since this worked perfectly. Now I understand the importance of this in a larger application.

I had the most trouble with the quota logic. My first attempt added up the number of cylinders per month, which gave me the wrong results. Once I added the month verification to the condition, it all worked well. It caused me to try every aspect and verify that the logic is correct, rather than assuming that because there are no compile errors, then it is correct.

Another good lesson was the NullPointerException. Always check for null when calling any method on a string. The safe way to do this is to use both null || isEmpty().

Overall, I'm happy with the results of Milestone 1. A total of 2 test scenarios were achieved successfully, and the code is correct and reflects the encapsulation, abstraction, inheritance and polymorphism concepts. In Milestone 2, I will create the Swing GUI according to the class structure that I have already created.

8. Refrences

Bloch, J., 2018. Effective Java. 3rd edn. Boston: Addison-Wesley.

Oracle, 2024a. Java SE 21 API: ArrayList. [online] Available at: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/ArrayList.html [Accessed 31 July 2026].

Oracle, 2024b. The Java Tutorials: Learning the Java Language. [online] Available at: https://docs.oracle.com/javase/tutorial/java/ [Accessed 31 July 2026].

Oracle, 2024c. Creating a GUI with Swing. [online] Available at: https://docs.oracle.com/javase/tutorial/uiswing/ [Accessed 31 July 2026].

Oracle, 2024d. Java Platform, Standard Edition Documentation. [online] Available at: https://docs.oracle.com/en/java/javase/ [Accessed 31 July 2026].

9. Appendix

9.1 LPGCylinder

```
public abstract class LPGCylinder {
    private String cylinderId;
    private String cylinderType;
    private double basePrice;
    private double weight;

    // constructor
    public LPGCylinder(String cylinderId, String cylinderType,
                    double basePrice, double weight) {
        setCylinderId(cylinderId);
        setCylinderType(cylinderType);
        setBasePrice(basePrice);
        setWeight(weight);
    }

    // getter for cylinder id
    public String getCylinderId() {
        return cylinderId;
    }

    // setter for cylinder id
    public void setCylinderId(String cylinderId) {
```

```
        if (cylinderId == null || cylinderId.trim().isEmpty()) {
            throw new IllegalArgumentException("Cylinder ID cannot be empty.");
        }

        this.cylinderId = cylinderId;
    }

    // getter for cylinder type
    public String getCylinderType() {
        return cylinderType;
    }

    // setter for cylinder type
    public void setCylinderType(String cylinderType) {
        if (cylinderType == null || cylinderType.trim().isEmpty()) {
            throw new IllegalArgumentException("Cylinder type cannot be empty.");
        }

        this.cylinderType = cylinderType;
    }

    // getter for price
    public double getBasePrice() {
        return basePrice;
```

```
    }

    // setter for price
    public void setBasePrice(double basePrice) {
        if (basePrice <= 0) {
            throw new IllegalArgumentException("Price must be greater than zero.");
        }

        this.basePrice = basePrice;
    }

    // getter for weight
    public double getWeight() {
        return weight;
    }

    // setter for weight
    public void setWeight(double weight) {
        if (weight <= 0) {
            throw new IllegalArgumentException("Weight must be greater than zero.");
        }

        this.weight = weight;
    }
```

```
    // child classes must calculate final price
    public abstract double calculateFinalPrice();

    // child classes must display their details
    public abstract String display();

    @Override
    public String toString() {
        return display();
    }
}
```

9.2 Domestic Cylinder

```
public class DomesticCylinder extends LPGCylinder {
    private String bookingId;
    private String month;
    private double subsidyAmount;
    private String citizenshipNumber;

    // constructor
    public DomesticCylinder(String cylinderId, String bookingId,
                            String month, double basePrice,
                            double weight, double subsidyAmount,
                            String citizenshipNumber) {
```

```
        super(cylinderId, "Domestic", basePrice, weight);

        setBookingId(bookingId);
        setMonth(month);
        setSubsidyAmount(subsidyAmount);
        setCitizenshipNumber(citizenshipNumber);
    }

    // getter for booking id
    public String getBookingId() {
        return bookingId;
    }

    // setter for booking id
    public void setBookingId(String bookingId) {
        if (bookingId == null || bookingId.trim().isEmpty()) {
            throw new IllegalArgumentException("Booking ID cannot be empty.");
        }

        this.bookingId = bookingId;
    }

    // getter for month
    public String getMonth() {
```

```
        return month;
    }

    // setter for month
    public void setMonth(String month) {
        if (month == null || month.trim().isEmpty()) {
            throw new IllegalArgumentException("Month cannot be empty.");
        }

        this.month = month;
    }

    // getter for subsidy
    public double getSubsidyAmount() {
        return subsidyAmount;
    }

    // setter for subsidy
    public void setSubsidyAmount(double subsidyAmount) {
        if (subsidyAmount < 0) {
            throw new IllegalArgumentException("Subsidy cannot be negative.");
        }

        if (subsidyAmount > getBasePrice()) {
```

```
            throw new IllegalArgumentException(
                    "Subsidy cannot be greater than the base price.");
        }

        this.subsidyAmount = subsidyAmount;
    }

    // getter for citizenship number
    public String getCitizenshipNumber() {
        return citizenshipNumber;
    }

    // setter for citizenship number
    public void setCitizenshipNumber(String citizenshipNumber) {
        if (citizenshipNumber == null ||
                !citizenshipNumber.matches("[0-9]{12}")) {
            throw new IllegalArgumentException(
                    "Citizenship number must contain exactly 12 digits.");
        }

        this.citizenshipNumber = citizenshipNumber;
    }

    // calculate price after subsidy
```

```
    @Override
    public double calculateFinalPrice() {
        return getBasePrice() - subsidyAmount;
    }

    // display domestic cylinder details
    @Override
    public String display() {
        return "Domestic Cylinder" +
                "\nCylinder ID: " + getCylinderId() +
                "\nBooking ID: " + bookingId +
                "\nMonth: " + month +
                "\nPrice: " + getBasePrice() +
                "\nWeight: " + getWeight() + " kg" +
                "\nSubsidy: " + subsidyAmount +
                "\nFinal Price: " + calculateFinalPrice() +
                "\nCitizenship Number: " + citizenshipNumber +
                "\n----------------------------";
    }
}
```

9.3 Commercial Cylinders

```
public class CommercialCylinder extends LPGCylinder {
    private String bookingId;
    private String month;
    private int quantity;
    private String businessLicense;

    // constructor
    public CommercialCylinder(String cylinderId, String bookingId,
                              String month, double basePrice,
                              double weight, int quantity,
                              String businessLicense) {
        super(cylinderId, "Commercial", basePrice, weight);

        setBookingId(bookingId);
        setMonth(month);
        setQuantity(quantity);
        setBusinessLicense(businessLicense);
    }

    // getter for booking id
    public String getBookingId() {
        return bookingId;
    }
```

```
    // setter for booking id
    public void setBookingId(String bookingId) {
        if (bookingId == null || bookingId.trim().isEmpty()) {
            throw new IllegalArgumentException("Booking ID cannot be empty.");
        }

        this.bookingId = bookingId;
    }

    // getter for month
    public String getMonth() {
        return month;
    }

    // setter for month
    public void setMonth(String month) {
        if (month == null || month.trim().isEmpty()) {
            throw new IllegalArgumentException("Month cannot be empty.");
        }

        this.month = month;
    }
```

```
    // getter for quantity
    public int getQuantity() {
        return quantity;
    }

    // setter for quantity
    public void setQuantity(int quantity) {
        if (quantity <= 0) {
            throw new IllegalArgumentException(
                    "Quantity must be greater than zero.");
        }

        this.quantity = quantity;
    }

    // getter for business license
    public String getBusinessLicense() {
        return businessLicense;
    }

    // setter for business license
    public void setBusinessLicense(String businessLicense) {
        if (businessLicense == null ||
                businessLicense.trim().isEmpty()) {
```

```
            throw new IllegalArgumentException(
                    "Business license cannot be empty.");
        }

        this.businessLicense = businessLicense;
    }

    // calculate total price after bulk discount
    @Override
    public double calculateFinalPrice() {
        double totalPrice = getBasePrice() * quantity;
        double discount = 0;

        if (quantity >= 10) {
            discount = totalPrice * 0.05;
        } else if (quantity >= 5) {
            discount = totalPrice * 0.03;
        }

        return totalPrice - discount;
    }

    // display commercial cylinder details
    @Override
```

```
    public String display() {
        return "Commercial Cylinder" +
                "\nCylinder ID: " + getCylinderId() +
                "\nBooking ID: " + bookingId +
                "\nMonth: " + month +
                "\nPrice Per Cylinder: " + getBasePrice() +
                "\nWeight: " + getWeight() + " kg" +
                "\nQuantity: " + quantity +
                "\nBusiness License: " + businessLicense +
                "\nTotal Price After Discount: " + calculateFinalPrice() +
                "\n----------------------------";
    }
}
```

9.4 NOCApp

```
import javax.swing.*;
import javax.swing.filechooser.FileNameExtensionFilter;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.io.*;
import java.util.ArrayList;


public class NOCApp implements ActionListener {
    private JFrame frame;


    private JComboBox<String> monthCombo;
    private JComboBox<String> typeCombo;
    private JComboBox<String> weightCombo;


    private JTextField txtSubsidy;
    private JTextField txtCitizenship;
    private JTextField txtLicense;
    private JTextField txtBookingId;
    private JTextField txtCylinderId;
    private JTextField txtPrice;
    private JTextField txtQuantity;
    private JTextField txtSearchId;
```

```
    private JTextArea outputArea;

    private JButton addDomesticButton;
    private JButton addCommercialButton;
    private JButton subsidyButton;
    private JButton discountButton;
    private JButton displayButton;
    private JButton identifyButton;
    private JButton clearButton;
    private JButton exportButton;
    private JButton loadButton;

    private ArrayList<LPGCylinder> cylinders;

    private String[] months =
        {
            "Baishakh",
            "Jestha",
            "Ashadh",
            "Shrawan",
            "Bhadra",
            "Ashwin",
            "Kartik",
```

```
            "Mangsir",
            "Poush",
            "Magh",
            "Falgun",
            "Chaitra"
        };

    private String[] weights =
        {
            "6.0",
            "12.5",
            "14.2",
            "19.0"
        };

    public NOCApp() {
        cylinders = new ArrayList<LPGCylinder>();

        frame = new JFrame("NOC LPG Cylinder Management");

        frame.setSize(950, 650);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(null);
```

```
        createGUI();

        frame.setVisible(true);
    }

    public void createGUI() {
        JPanel mainPanel = new JPanel(new BorderLayout(10, 10));

        // input panel
        JPanel inputPanel = new JPanel(new GridLayout(0, 2, 5, 5));

        inputPanel.setBorder(
                BorderFactory.createTitledBorder("Cylinder Details"));

        monthCombo = new JComboBox<String>(months);

        typeCombo = new JComboBox<String>(
                new String[]{"Domestic", "Commercial"});

        weightCombo = new JComboBox<String>(weights);

        txtSubsidy = new JTextField();
        txtCitizenship = new JTextField();
        txtLicense = new JTextField();
```

```
txtBookingId = new JTextField();
txtCylinderId = new JTextField();
txtPrice = new JTextField();
txtQuantity = new JTextField();
txtSearchId = new JTextField();

inputPanel.add(new JLabel("Month:"));
inputPanel.add(monthCombo);

inputPanel.add(new JLabel("Cylinder Type:"));
inputPanel.add(typeCombo);

inputPanel.add(new JLabel("Cylinder Weight (kg):"));
inputPanel.add(weightCombo);

inputPanel.add(new JLabel("Cylinder ID:"));
inputPanel.add(txtCylinderId);

inputPanel.add(new JLabel("Booking ID:"));
inputPanel.add(txtBookingId);

inputPanel.add(new JLabel("Price:"));
inputPanel.add(txtPrice);
```

```
        inputPanel.add(new JLabel("Quantity:"));
        inputPanel.add(txtQuantity);

        inputPanel.add(new JLabel("Subsidy Amount:"));
        inputPanel.add(txtSubsidy);

        inputPanel.add(new JLabel("Citizenship Number:"));
        inputPanel.add(txtCitizenship);

        inputPanel.add(new JLabel("Business License Number:"));
        inputPanel.add(txtLicense);

        inputPanel.add(new JLabel("Search Cylinder ID:"));
        inputPanel.add(txtSearchId);


        // button panel
        JPanel buttonPanel = new JPanel(new GridLayout(0, 2, 5, 5));

        buttonPanel.setBorder(
                BorderFactory.createTitledBorder("Actions"));

        addDomesticButton =
                new JButton("Add Domestic Cylinder");
```

```
addCommercialButton =
        new JButton("Add Commercial Cylinder");

subsidyButton =
        new JButton("Calculate Price After Subsidy");

discountButton =
        new JButton("Calculate Bulk Discount");

displayButton =
        new JButton("Display All");

identifyButton =
        new JButton("Identify Cylinder Type");

clearButton =
        new JButton("Clear");

exportButton =
        new JButton("Export to File");

loadButton =
        new JButton("Load From File");
```

```
buttonPanel.add(addDomesticButton);
buttonPanel.add(addCommercialButton);
buttonPanel.add(subsidyButton);
buttonPanel.add(discountButton);
buttonPanel.add(displayButton);
buttonPanel.add(identifyButton);
buttonPanel.add(clearButton);
buttonPanel.add(exportButton);
buttonPanel.add(loadButton);


// output area
outputArea = new JTextArea();

outputArea.setEditable(false);

outputArea.setLineWrap(true);

outputArea.setWrapStyleWord(true);

JScrollPane scrollPane =
        new JScrollPane(outputArea);
```

```
scrollPane.setBorder(
        BorderFactory.createTitledBorder("Output"));


JPanel topPanel = new JPanel(new BorderLayout(10, 10));

topPanel.add(inputPanel, BorderLayout.CENTER);

topPanel.add(buttonPanel, BorderLayout.SOUTH);


mainPanel.add(topPanel, BorderLayout.NORTH);

mainPanel.add(scrollPane, BorderLayout.CENTER);


frame.add(mainPanel);


// adding action listeners
addDomesticButton.addActionListener(this);

addCommercialButton.addActionListener(this);
```

```
        subsidyButton.addActionListener(this);

        discountButton.addActionListener(this);

        displayButton.addActionListener(this);

        identifyButton.addActionListener(this);

        clearButton.addActionListener(this);

        exportButton.addActionListener(this);

        loadButton.addActionListener(this);
    }


    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == addDomesticButton) {
            addDomesticCylinder();
        } else if (e.getSource() == addCommercialButton) {
            addCommercialCylinder();
        } else if (e.getSource() == subsidyButton) {
            calculateSubsidy();
```

```
        } else if (e.getSource() == discountButton) {
            calculateDiscount();
        } else if (e.getSource() == displayButton) {
            displayAll();
        } else if (e.getSource() == identifyButton) {
            identifyCylinder();
        } else if (e.getSource() == clearButton) {
            clearFields();
        } else if (e.getSource() == exportButton) {
            exportFile();
        } else if (e.getSource() == loadButton) {
            loadFile();
        }
    }


    // add domestic cylinder
    public void addDomesticCylinder() {
        try {
            String cylinderId =
                    txtCylinderId.getText().trim();

            String bookingId =
                    txtBookingId.getText().trim();
```

```
String month =
        monthCombo.getSelectedItem().toString();

String priceText =
        txtPrice.getText().trim();

String weightText =
        weightCombo.getSelectedItem().toString();

String subsidyText =
        txtSubsidy.getText().trim();

String citizenship =
        txtCitizenship.getText().trim();



if (cylinderId.isEmpty() ||
        bookingId.isEmpty() ||
        priceText.isEmpty() ||
        subsidyText.isEmpty() ||
        citizenship.isEmpty()) {
    throw new IllegalArgumentException(
            "Please fill all required domestic fields.");
```

```
        }



        // check cylinder id
        if (!cylinderId.matches("NOC-[0-9]+")) {
            throw new IllegalArgumentException(
                    "Cylinder ID must be like NOC-001.");
        }



        // check duplicate ids
        if (isDuplicateCylinderId(cylinderId)) {
            throw new IllegalArgumentException(
                    "Cylinder ID already exists.");
        }



        if (isDuplicateBookingId(bookingId)) {
            throw new IllegalArgumentException(
                    "Booking ID already exists.");
        }



        // check citizenship
```

```
if (!citizenship.matches("[0-9]{12}")) {
    throw new IllegalArgumentException(
            "Citizenship number must contain exactly 12 digits.");
}



// check monthly quota
if (checkDomesticQuota(citizenship, month)) {
    throw new IllegalArgumentException(
            "Monthly quota exceeded. Maximum is 2 cylinders.");
}



double price =
        Double.parseDouble(priceText);

double weight =
        Double.parseDouble(weightText);

double subsidy =
        Double.parseDouble(subsidyText);



if (price <= 0 || weight <= 0) {
```

```
        throw new IllegalArgumentException(
                "Price and weight must be greater than zero.");
    }



    if (subsidy < 0) {
        throw new IllegalArgumentException(
                "Subsidy cannot be negative.");
    }



    if (subsidy > price) {
        throw new IllegalArgumentException(
                "Subsidy cannot be greater than price.");
    }



    DomesticCylinder domestic =
            new DomesticCylinder(
                    cylinderId,
                    bookingId,
                    month,
                    price,
                    weight,
```

```
                    subsidy,
                    citizenship);


            cylinders.add(domestic);


            outputArea.append(
                "Domestic cylinder added successfully.\n");

            outputArea.append(
                domestic.display() + "\n");


            JOptionPane.showMessageDialog(
                frame,
                "Domestic cylinder added successfully.");
        } catch (NumberFormatException ex) {
            showError("Please enter valid numbers.");
        } catch (IllegalArgumentException ex) {
            showError(ex.getMessage());
        }
    }
```

```
// add commercial cylinder
public void addCommercialCylinder() {
    try {
        String cylinderId =
                txtCylinderId.getText().trim();

        String bookingId =
                txtBookingId.getText().trim();

        String month =
                monthCombo.getSelectedItem().toString();

        String priceText =
                txtPrice.getText().trim();

        String weightText =
                weightCombo.getSelectedItem().toString();

        String quantityText =
                txtQuantity.getText().trim();

        String license =
                txtLicense.getText().trim();
```

```
if (cylinderId.isEmpty() ||
        bookingId.isEmpty() ||
        priceText.isEmpty() ||
        quantityText.isEmpty() ||
        license.isEmpty()) {
    throw new IllegalArgumentException(
            "Please fill all required commercial fields.");
}



// check cylinder id
if (!cylinderId.matches("NOC-[0-9]+")) {
    throw new IllegalArgumentException(
            "Cylinder ID must be like NOC-001.");
}



// check duplicate ids
if (isDuplicateCylinderId(cylinderId)) {
    throw new IllegalArgumentException(
            "Cylinder ID already exists.");
}
```

```
if (isDuplicateBookingId(bookingId)) {
    throw new IllegalArgumentException(
            "Booking ID already exists.");
}



double price =
        Double.parseDouble(priceText);

double weight =
        Double.parseDouble(weightText);

int quantity =
        Integer.parseInt(quantityText);



if (price <= 0 || weight <= 0) {
    throw new IllegalArgumentException(
            "Price and weight must be greater than zero.");
}
```

```
if (quantity <= 0) {
    throw new IllegalArgumentException(
            "Quantity must be greater than zero.");
}



CommercialCylinder commercial =
        new CommercialCylinder(
                cylinderId,
                bookingId,
                month,
                price,
                weight,
                quantity,
                license);



cylinders.add(commercial);



outputArea.append(
        "Commercial cylinder added successfully.\n");

outputArea.append(
```

```
                commercial.display() + "\n");



            JOptionPane.showMessageDialog(
                    frame,
                    "Commercial cylinder added successfully.");
        } catch (NumberFormatException ex) {
            showError("Please enter valid numbers.");
        } catch (IllegalArgumentException ex) {
            showError(ex.getMessage());
        }
    }



    // calculate domestic price after subsidy
    public void calculateSubsidy() {
        String id =
                txtSearchId.getText().trim();



        if (id.isEmpty()) {
            showError(
                    "Please enter a Cylinder ID.");
            return;
```

```
    }


    LPGCylinder cylinder =
            findCylinder(id);



    if (cylinder == null) {
        showError(
                "Cylinder ID not found.");
        return;
    }


    // check type before casting
    if (cylinder instanceof DomesticCylinder) {
        DomesticCylinder domestic =
                (DomesticCylinder) cylinder;



        double finalPrice =
                domestic.calculateFinalPrice();
```

```
            outputArea.append(
                    "Price after subsidy: " +
                            finalPrice + "\n");



            JOptionPane.showMessageDialog(
                    frame,
                    "Price after subsidy: " +
                            finalPrice);
        } else if (cylinder instanceof CommercialCylinder) {
            showError(
                    "This is a commercial cylinder. " +
                            "Please use Calculate Bulk Discount.");
        }
    }



    // calculate commercial bulk discount
    public void calculateDiscount() {
        String id =
                txtSearchId.getText().trim();



        if (id.isEmpty()) {
```

```
        showError(
                "Please enter a Cylinder ID.");
        return;
    }


    LPGCylinder cylinder =
            findCylinder(id);



    if (cylinder == null) {
        showError(
                "Cylinder ID not found.");
        return;
    }



    // check type before casting
    if (cylinder instanceof CommercialCylinder) {
        CommercialCylinder commercial =
                (CommercialCylinder) cylinder;



        double finalPrice =
```

```
                commercial.calculateFinalPrice();



            outputArea.append(
                "Total price after bulk discount: " +
                    finalPrice + "\n");



            JOptionPane.showMessageDialog(
                frame,
                "Total price after bulk discount: " +
                    finalPrice);
        } else if (cylinder instanceof DomesticCylinder) {
            showError(
                "This is a domestic cylinder. " +
                    "Please use Calculate Price After Subsidy.");
        }
    }



    // display all cylinders
    public void displayAll() {
        if (cylinders.size() == 0) {
            outputArea.setText(
```

```
                "No cylinder records available.");
            return;
        }


        outputArea.setText(
                "===== ALL CYLINDER DETAILS =====\n\n");


        for (int i = 0;
             i < cylinders.size();
             i++) {
            LPGCylinder cylinder =
                    cylinders.get(i);


            outputArea.append(
                    cylinder.display() + "\n\n");
        }
    }


    // identify cylinder type
    public void identifyCylinder() {
```

```
String id =
        txtSearchId.getText().trim();



if (id.isEmpty()) {
    showError(
            "Please enter a Cylinder ID.");
    return;
}



LPGCylinder cylinder =
        findCylinder(id);



if (cylinder == null) {
    showError(
            "Cylinder ID not found.");
    return;
}



if (cylinder instanceof DomesticCylinder) {
    outputArea.append(
```

```
                id + " is a Domestic Cylinder.\n");



            JOptionPane.showMessageDialog(
                frame,
                "Domestic Cylinder");
        } else if (cylinder instanceof CommercialCylinder) {
            outputArea.append(
                id + " is a Commercial Cylinder.\n");



            JOptionPane.showMessageDialog(
                frame,
                "Commercial Cylinder");
        } else {
            showError(
                "Unknown cylinder type.");
        }
    }



    // clear all fields
    public void clearFields() {
        txtSubsidy.setText("");
```

```
        txtCitizenship.setText("");

        txtLicense.setText("");

        txtBookingId.setText("");

        txtCylinderId.setText("");

        txtPrice.setText("");

        txtQuantity.setText("");

        txtSearchId.setText("");

        monthCombo.setSelectedIndex(0);

        typeCombo.setSelectedIndex(0);

        weightCombo.setSelectedIndex(0);

        outputArea.setText("");
    }
```

```
// check duplicate cylinder id
public boolean isDuplicateCylinderId(
        String id) {
    for (int i = 0;
         i < cylinders.size();
         i++) {
        if (cylinders.get(i)
                .getCylinderId()
                .equalsIgnoreCase(id)) {
            return true;
        }
    }

    return false;
}


// check duplicate booking id
public boolean isDuplicateBookingId(
        String id) {
    for (int i = 0;
         i < cylinders.size();
         i++) {
```

```
        LPGCylinder cylinder =
                cylinders.get(i);



        if (cylinder instanceof DomesticCylinder) {
            DomesticCylinder domestic =
                    (DomesticCylinder) cylinder;



            if (domestic.getBookingId()
                    .equalsIgnoreCase(id)) {
                return true;
            }
        } else if (cylinder instanceof CommercialCylinder) {
            CommercialCylinder commercial =
                    (CommercialCylinder) cylinder;



            if (commercial.getBookingId()
                    .equalsIgnoreCase(id)) {
                return true;
            }
        }
    }
```

```
        return false;
    }


    // check domestic monthly quota
    public boolean checkDomesticQuota(
            String citizenship,
            String month) {
        int count = 0;



        for (int i = 0;
             i < cylinders.size();
             i++) {
            LPGCylinder cylinder =
                    cylinders.get(i);



            if (cylinder instanceof DomesticCylinder) {
                DomesticCylinder domestic =
                        (DomesticCylinder) cylinder;
```

```
            if (domestic.getCitizenshipNumber()
                    .equals(citizenship)
                    &&
                    domestic.getMonth()
                            .equals(month)) {
                count++;
            }
        }
    }


    if (count >= 2) {
        return true;
    }


    return false;
}


// find cylinder using id
public LPGCylinder findCylinder(
        String id) {
    for (int i = 0;
```

```
            i < cylinders.size();
            i++) {
            LPGCylinder cylinder =
                    cylinders.get(i);



            if (cylinder.getCylinderId()
                    .equalsIgnoreCase(id)) {
                return cylinder;
            }
        }



        return null;
    }



    // export data to file
    public void exportFile() {
        if (cylinders.size() == 0) {
            showError(
                    "There is no data to export.");
            return;
        }
```

```
JFileChooser chooser =
        new JFileChooser();


chooser.setDialogTitle(
        "Save Cylinder Data");


chooser.setFileFilter(
        new FileNameExtensionFilter(
                "Text Files",
                "txt"));


int result =
        chooser.showSaveDialog(frame);


if (result != JFileChooser.APPROVE_OPTION) {
    return;
}
```

```
File file =
        chooser.getSelectedFile();



if (!file.getName()
        .toLowerCase()
        .endsWith(".txt")) {
    file = new File(
            file.getAbsolutePath() + ".txt");
}



try {
    BufferedWriter writer =
            new BufferedWriter(
                    new FileWriter(file));



    for (int i = 0;
         i < cylinders.size();
         i++) {
        LPGCylinder cylinder =
                cylinders.get(i);
```

```
if (cylinder instanceof DomesticCylinder) {
    DomesticCylinder domestic =
            (DomesticCylinder) cylinder;



    writer.write(
            "Domestic|" +
                    domestic.getCylinderId() + "|" +
                    domestic.getBookingId() + "|" +
                    domestic.getMonth() + "|" +
                    domestic.getBasePrice() + "|" +
                    domestic.getWeight() + "|" +
                    domestic.getSubsidyAmount() + "|" +
                    domestic.getCitizenshipNumber());


    writer.newLine();
} else if (cylinder instanceof CommercialCylinder) {
    CommercialCylinder commercial =
            (CommercialCylinder) cylinder;



    writer.write(
```

```
                    "Commercial|" +
                            commercial.getCylinderId() + "|" +
                            commercial.getBookingId() + "|" +
                            commercial.getMonth() + "|" +
                            commercial.getBasePrice() + "|" +
                            commercial.getWeight() + "|" +
                            commercial.getQuantity() + "|" +
                            commercial.getBusinessLicense());

                writer.newLine();
            }
        }


        writer.close();


        JOptionPane.showMessageDialog(
                frame,
                "Data exported successfully.");
    } catch (IOException ex) {
        showError(
                "Error while exporting file.");
    }
```

```
    }


    // load data from file
    public void loadFile() {
        JFileChooser chooser =
                new JFileChooser();


        chooser.setDialogTitle(
                "Load Cylinder Data");


        chooser.setFileFilter(
                new FileNameExtensionFilter(
                        "Text Files",
                        "txt"));


        int result =
                chooser.showOpenDialog(frame);


        if (result != JFileChooser.APPROVE_OPTION) {
```

```
        return;
    }


    File file =
            chooser.getSelectedFile();



    try {
        BufferedReader reader =
                new BufferedReader(
                        new FileReader(file));



        cylinders.clear();



        String line;


        String allData = "";



        while ((line = reader.readLine()) != null) {
            allData =
```

```
            allData + line + "\n";



        String[] data =
            line.split("\\|");



        if (data[0].equals("Domestic")) {
            if (data.length == 8) {
                DomesticCylinder domestic =
                        new DomesticCylinder(
                                data[1],
                                data[2],
                                data[3],
                                Double.parseDouble(data[4]),
                                Double.parseDouble(data[5]),
                                Double.parseDouble(data[6]),
                                data[7]);



                cylinders.add(domestic);
            }
        } else if (data[0].equals("Commercial")) {
            if (data.length == 8) {
```

```
                CommercialCylinder commercial =
                        new CommercialCylinder(
                                data[1],
                                data[2],
                                data[3],
                                Double.parseDouble(data[4]),
                                Double.parseDouble(data[5]),
                                Integer.parseInt(data[6]),
                                data[7]);



                    cylinders.add(commercial);
                }
            }
        }


        reader.close();



        showLoadedData(
                file.getName(),
                allData);
```

```
            outputArea.append(
                    "File loaded successfully.\n");



            JOptionPane.showMessageDialog(
                    frame,
                    "Data loaded successfully.");
        } catch (IOException ex) {
            showError(
                    "Error while loading file.");
        } catch (NumberFormatException ex) {
            showError(
                    "Invalid number found in file.");
        } catch (IllegalArgumentException ex) {
            showError(
                    ex.getMessage());
        }
    }



    // show loaded file in a new window
    public void showLoadedData(
            String fileName,
```

```
    String data) {
JFrame newFrame =
        new JFrame(
                "Loaded Data - " +
                        fileName);



newFrame.setSize(
        600,
        400);



newFrame.setLocationRelativeTo(frame);



JTextArea area =
        new JTextArea();



area.setEditable(false);



area.setText(data);
```

```
        JScrollPane scroll =
                new JScrollPane(area);



        newFrame.add(scroll);



        newFrame.setVisible(true);
    }



    // show error message
    public void showError(
            String message) {
        outputArea.append(
                "Error: " +
                        message +
                        "\n");



        JOptionPane.showMessageDialog(
                frame,
                message,
```

```
            "Error",
            JOptionPane.ERROR_MESSAGE);
    }


    // main method
    public static void main(String[] args) {
        new NOCApp();
    }
}
```